# Lab Manual 07
# Code Explanations

## Inter-Process Communication (IPC) – Part 2
## Message Queues & Shared Memory

---

**Course:** CL-2006 – Operating System Lab
**Instructor:** Yasir Arfat
**Semester:** Spring 2025

**Covers:** All 4 Example Codes from Lab Manual 07
Example 1: Message Queue Sender
Example 2: Message Queue Receiver
Example 3: Shared Memory Server (Writer)
Example 4: Shared Memory Client (Reader)

---

**Line-by-Line Explanation with Theory Concepts,**
**Memory Diagrams, API Tables, Output Traces, and Quiz Notes**

**How to use this document – Colour Guide:**

- **Yellow box** – line-by-line code explanation.
- **Green box** – OS theory concept from Lab Manual 07.
- **Red/Pink box** – WARNING / common mistake to avoid.
- **Blue box** – TIP / important quiz point.
- **Purple box** – memory / variable state / API diagram.
- **Light blue box** – step-by-step flow or timeline.
- **Orange box** – KEY CONCEPT summary.

# Contents

# 1 Background Theory – IPC Mechanisms Overview

Before diving into code, you must clearly understand the two new IPC mechanisms in Lab 07 and how they compare to pipes from Lab 06.

## 1.1 What Lab 07 Adds to Your IPC Knowledge

**IPC Journey So Far (Lab Manual 07, Recap Section):**

**Lab 06 covered:**

- **Unnamed Pipes** – parent-to-child, byte stream, unidirectional, auto-sync.
- **Named Pipes (FIFO)** – any two processes via filesystem path, byte stream.

**Lab 07 adds:**

- **Message Queues** – structured typed messages, message boundaries preserved, kernel-managed buffer, bidirectional via types.
- **Shared Memory** – fastest IPC, zero kernel copies, direct access to same physical memory, but requires manual synchronization.

## 1.2 The Critical Difference: Data Copies

**Why Shared Memory is the Fastest (Lab Manual 07, OS CONCEPT):**

**Pipes and Message Queues:** Data travels through the kernel buffer. This means **2 copies** happen:

1. Sender's address space → kernel buffer (copy 1).
2. Kernel buffer → receiver's address space (copy 2).

**Shared Memory:** The kernel maps the **same physical memory pages** into both processes' virtual address spaces. **Zero copies**. Both processes read and write directly to the same RAM location.

**Downside of Shared Memory:** No built-in synchronization. You must use semaphores to prevent race conditions (covered in a future lab).

## 1.3 IPC Mechanism Comparison Table

| Feature | Pipes (Lab 06) | Msg Queue (Lab 07) | Shared Mem (Lab 07) |
|---|---|---|---|
| Data format | Raw byte stream | Structured + typed | Raw memory |
| Message boundaries | No | Yes (discrete msgs) | No |
| Speed | Fast | Medium | **Fastest** (0 copies) |
| Synchronization | Built-in (blocking) | Built-in (blocking) | **NONE – manual** |
| Persistence | Dies with pipe | Until deleted | Until deleted |
| Data copies | 2 | 2 | **0** |
| Identifier | File descriptor | Queue ID (int) | Segment ID + pointer |
| Cleanup | Automatic | `msgctl(IPC_RMID)` | `shmctl(IPC_RMID)` |
| Shell inspect | — | `ipcs -q` | `ipcs -m` |

## 1.4 The IPC Key – ftok() Concept

**What is an IPC Key and why is ftok() needed?**

Both message queues and shared memory need a **unique integer key** so that two separate, unrelated processes can find the **same** queue or memory segment. Think of it like an apartment number – both the sender and receiver must know the same apartment number to communicate.

```
key_t key = ftok("somefile.c", 'b');
```

**How ftok() works:**

1. Takes a file path (`"somefile.c"`) and a project character (`'b'`).
2. Uses the file's **inode number** (a unique ID the kernel assigns to every file) combined with the project character to generate a unique integer.
3. As long as both sender and receiver call `ftok()` with the **same file path and same character**, they get the **same key**.
4. If the file does not exist, `ftok()` returns `-1` (error).

**Rule:** Both communicating processes MUST use the same `ftok()` arguments.

# 2 Example 1 – Message Queue Sender

**Purpose of this example:** Show how to create a message queue using System V API, set a message type, send messages in a loop reading from user input, and clean up the queue when done. This is the "sender" side – it creates the queue and owns it.

## 2.1 Full Source Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/types.h>
#include <sys/msg.h>

struct msgbuf {
    long mtype;
    char msgtxt[200];
};

int main(void) {
    struct msgbuf msg;
    int msgid;
    key_t key;

    key = ftok("message_send.c", 'b');
    if (key == -1) { perror("ftok"); exit(1); }

    msgid = msgget(key, 0644 | IPC_CREAT);
    if (msgid == -1) { perror("msgget"); exit(1); }

    printf("message_send [INFO] Queue ID: %d\n", msgid);
    printf("message_send [PROMPT] Enter messages (Ctrl+D to stop)
       :\n");

    msg.mtype = 1;
    while (fgets(msg.msgtxt, 200, stdin)) {
        if (msgsnd(msgid, &msg, sizeof(msg), 0) == -1) {
            perror("msgsnd");
            exit(1);
        }
    }

    msgctl(msgid, IPC_RMID, NULL);
    printf("message_send [INFO] Queue deleted.\n");
    return 0;
}
```

Listing 1: Example 1: message_send.c – Message Queue Sender

## 2.2 Header Files – Line-by-Line

```
#include <stdio.h>
```

**Line 1 – `#include <stdio.h>`:**
Standard Input/Output library. Provides:

- `printf()` – print formatted text to the terminal.
- `fgets()` – read a line of text from a file or stdin.
- `perror()` – print a system error message to stderr.

`fgets()` is used in the sender loop to read user input line by line.

```
#include <stdlib.h>
```

**Line 2 – `#include <stdlib.h>`:**
Standard library. Provides `exit(1)` which immediately terminates the process with an error code when a system call fails. Used in every error-checking block in this file.

```
#include <string.h>
```

**Line 3 – `#include <string.h>`:**
String handling library. In this example it is included for completeness (for `strcpy`, `strlen` etc. that might be needed if you extend the code). `fgets()` itself does not require this header but `string.h` is commonly included when working with character arrays.

```
#include <sys/ipc.h>
```

**Line 4 – `#include <sys/ipc.h>`:**
IPC (Inter-Process Communication) header. Provides:

- `ftok()` – the function that generates a unique IPC key.
- The `key_t` data type (a typedef for an integer used as an IPC key).
- The `IPC_CREAT` flag (used with `msgget()` to create a new queue if it does not exist).
- The `IPC_RMID` command (used with `msgctl()` to delete the queue).

**This header is required for ALL System V IPC (both message queues and shared memory).**

```
#include <sys/types.h>
```

**Line 5 – `#include <sys/types.h>`:**
Provides fundamental system data types. Ensures `key_t` and other IPC types are correctly defined across different Unix/Linux systems. Many system headers pull this in automatically, but including it explicitly is good practice for portability.

```
#include <sys/msg.h>
```

**Line 6 – `#include <sys/msg.h>`:**
Message queue specific header. This is the most important header for this example. It provides all four message queue system calls:

- `msgget()` – create or open a message queue.
- `msgsnd()` – send a message to the queue.
- `msgrcv()` – receive a message from the queue.
- `msgctl()` – control/delete the queue.

**Without this header, none of the message queue functions will compile.**

## 2.3 The Message Structure

```
struct msgbuf {
    long mtype;
    char msgtxt[200];
};
```

**Lines 8–11 – `struct msgbuf` (The Message Structure):**
This is the blueprint for every message sent through the queue.

- `long mtype` – **MUST be the first field and MUST be a positive integer** (greater than 0). This is enforced by the kernel. The type number acts like a label – the receiver can choose to receive only messages of a specific type. In this example, all messages use type 1.
  **If mtype $\leq$ 0, msgsnd() will fail with EINVAL error.**
- `char msgtxt[200]` – this is your actual data payload. It is a 200-byte character array that holds the text of the message. You can name this field anything and use any data type – it does not have to be a `char` array. You could use `int`, `float`, or a nested struct here.

**Total size of this struct:** `sizeof(long)` + `sizeof(char[200])` = 8 + 200 = 208 bytes.

**Theory: Why must the first field be `long mtype`?**

The System V message queue API is designed so that the kernel reads the first `sizeof(long)` bytes of your message struct to determine its type. This is a **fixed contract** between your program and the kernel. The rest of the struct (after `mtype`) is your data and the kernel treats it as opaque bytes – it does not care what is in there. This design allows the receiver to filter messages by type without reading the full message content.

## 2.4 main() – Variable Declarations

```
    struct msgbuf msg;
    int msgid;
    key_t key;
```

**Lines 14–16 – Variable declarations:**

- `struct msgbuf msg` – an instance of the message struct declared above. This is the actual message object that will be filled with data and sent. It lives on the stack.
- `int msgid` – will hold the message queue ID returned by `msgget()`. This integer is the "handle" used in all subsequent calls (`msgsnd()`, `msgctl()`) to refer to this specific queue.
- `key_t key` – will hold the IPC key generated by `ftok()`. `key_t` is a typedef (usually an `int`) defined in `<sys/ipc.h>`.

## 2.5 Step 1 – Generating the IPC Key

```
    key = ftok("message_send.c", 'b');
    if (key == -1) { perror("ftok"); exit(1); }
```

**Lines 18–19 – `ftok()` call with error checking:**
`ftok("message_send.c", 'b')`:

- Argument 1: `"message_send.c"` – a file that MUST exist on disk. `ftok()` reads this file's inode number from the filesystem. If the file does not exist, `ftok()` returns `-1`.
- Argument 2: `'b'` – a project identifier character. It is combined with the inode number to generate the key. Both sender and receiver must use the same character.
- **Returns:** a `key_t` integer (e.g., `12345678`).

**Error check:** `if (key == -1)` – if `ftok()` failed (file not found, permission denied), print the error and exit immediately. `perror("ftok")` prints: `ftok: No such file or directory` (or similar).

**WARNING – The file must exist!**
The file `"message_send.c"` must physically exist when you run the program. Since this is the source file itself, it will exist as long as you run the compiled binary from the same directory. Both sender and receiver must use the same file path and the same project character (`'b'`) to get the same key. If they use different values, they will get different keys and connect to different queues.

## 2.6 Step 2 – Creating the Message Queue

```
    msgid = msgget(key, 0644 | IPC_CREAT);
    if (msgid == -1) { perror("msgget"); exit(1); }
```

**Lines 21–22 – `msgget()` – Create or open the queue:**
`msgget(key, 0644 | IPC_CREAT)`:

- **Argument 1:** `key` – the IPC key generated by `ftok()`. The kernel searches for an existing queue with this key. If found and `IPC_CREAT` is set, it opens the existing one. If not found, it creates a new one.
- **Argument 2:** `0644 | IPC_CREAT`:
  - `0644` – Unix-style permission bits (octal). Owner: read+write (6 = rw); Group: read-only (4 = r); Others: read-only (4 = r). Written in octal like file permissions (`chmod`).
  - `IPC_CREAT` – a flag (defined in `<sys/ipc.h>`) that tells the kernel: "create the queue if it does not already exist."
  - The `|` (bitwise OR) combines both into one integer argument.
- **Returns:** an integer **queue ID (msgid)** on success, or `-1` on failure.

**Error check:** If `msgid == -1`, the queue could not be created (permission problem, system limit reached). Print error and exit.

**What msgget() does internally:**

| Situation | What happens |
|---|---|
| Queue with this key exists | Opens existing queue, returns its ID |
| Queue does not exist + `IPC_CREAT` | Creates new queue, returns new ID |
| Queue does not exist, no `IPC_CREAT` | Returns -1 (ENOENT error) |
| System limit reached | Returns -1 (ENOSPC error) |

## 2.7 Step 3 – Sending Messages in a Loop

```
        printf("message_send [INFO] Queue ID: %d\n", msgid);
        printf("message_send [PROMPT] Enter messages (Ctrl+D to stop)
           :\n");
```

**Lines 24–25 – Informational output:**
These two `printf()` calls simply inform the user:

- The queue ID (`msgid`) that was assigned. This is useful for debugging – you can verify the same ID appears in the receiver.
- Instructions: type messages and press Ctrl+D when done. Ctrl+D sends an EOF (End Of File) signal to `stdin`, which causes `fgets()` to return `NULL`, ending the loop.

```
        msg.mtype = 1;
```

**Line 27 – Setting the message type:**
Before the loop, the message type is set to `1`. All messages sent in this loop will have `mtype = 1`. This is done outside the loop for efficiency – the type does not change between messages, so there is no need to set it every iteration.

The receiver will call `msgrcv(..., 1, ...)` to specifically receive only type-1 messages. This is how type-based filtering works.

```
        while (fgets(msg.msgtxt, 200, stdin)) {
```

**Line 28 – `fgets()` as the loop condition:**
`fgets(msg.msgtxt, 200, stdin)`:

- **Argument 1:** `msg.msgtxt` – the destination buffer. Text read from stdin is stored directly into the message struct's data field.
- **Argument 2:** `200` – maximum bytes to read (including the null terminator `'\0'`). This prevents buffer overflow.
- **Argument 3:** `stdin` – read from standard input (keyboard).
- **Returns:** a pointer to `msg.msgtxt` on success, or `NULL` when EOF is reached (user presses Ctrl+D) or an error occurs.

The `while` loop continues as long as `fgets()` returns a non-NULL pointer (i.e., as long as there is more user input). When Ctrl+D is pressed, `fgets()` returns `NULL` and the loop exits.

**Note:** `fgets()` includes the newline character (`'\n'`) at the end of the string if the user pressed Enter. So `msg.msgtxt` might be `"hello\n"`.

```
            if (msgsnd(msgid, &msg, sizeof(msg), 0) == -1) {
                perror("msgsnd");
                exit(1);
            }
```

**Lines 29–32 – `msgsnd()` – Send one message:**
`msgsnd(msgid, &msg, sizeof(msg), 0)`:

- **Argument 1:** `msgid` – the queue ID returned by `msgget()`. Identifies which queue to send to.
- **Argument 2:** `&msg` – a pointer to the message struct. The kernel copies data from this address into the kernel's queue buffer.
- **Argument 3:** `sizeof(msg)` – the size of the entire struct in bytes (208 bytes in this case: 8 for `long` + 200 for `char[200]`). The kernel copies exactly this many bytes.
- **Argument 4:** `0` – flags. `0` means blocking mode: if the queue is full, `msgsnd()` will block (wait) until space is available. The alternative flag `IPC_NOWAIT` would return immediately with an error if the queue is full.
- **Returns:** `0` on success, `-1` on failure.

After `msgsnd()` returns, the message is safely in the kernel's buffer. The sender can reuse or modify `msg` – the kernel has its own copy.

**Theory: What happens inside the kernel when msgsnd() is called?**

1. The kernel copies the message struct from the sender's address space into a kernel buffer (this is Data Copy #1).
2. The kernel appends this message to the queue's linked list in sorted order by type.
3. If any process is blocked in `msgrcv()` waiting for a message of this type, the kernel wakes it up.
4. `msgsnd()` returns 0 to the sender.

The receiver will then trigger Data Copy #2 (kernel → receiver's address space) when it calls `msgrcv()`. Total: 2 data copies for message queues.

## 2.8 Step 4 – Cleanup: Deleting the Queue

```
        msgctl(msgid, IPC_RMID, NULL);
        printf("message_send [INFO] Queue deleted.\n");
        return 0;
```

**Lines 34–36 – `msgctl()` – Delete the queue:**
`msgctl(msgid, IPC_RMID, NULL)`:

- **Argument 1:** `msgid` – the queue to operate on.
- **Argument 2:** `IPC_RMID` – the command. `IPC_RMID` means "Remove this IPC resource immediately." The kernel destroys the queue and frees all memory associated with it. Any unread messages in the queue are also destroyed.
- **Argument 3:** `NULL` – a pointer to a `struct msqid_ds` struct for `IPC_STAT` (get queue info) or `IPC_SET` (set permissions). For `IPC_RMID`, this is not needed, so `NULL` is passed.

After this call, the queue no longer exists. If the receiver tries to call `msgrcv()` after this, it will get an error.

**WARNING – Message Queues Persist in the Kernel!**
Unlike pipes that disappear automatically, message queues **survive until explicitly deleted** – even if both programs exit without calling `msgctl(IPC_RMID)`. An orphaned queue occupies kernel memory indefinitely.

**To check for orphaned queues:** `ipcs -q`
**To delete one manually:** `ipcrm -q <msgid>`

Always call `msgctl(msgid, IPC_RMID, NULL)` in your cleanup code.

## 2.9 Complete Expected Output – Example 1

```
message_send [INFO] Queue ID: 0
message_send [PROMPT] Enter messages (Ctrl+D to stop):
hello This is Yasir          <- user types this
this is second message       <- user types this
^D                           <- user presses Ctrl+D
message_send [INFO] Queue deleted.
```

# 3 Example 2 – Message Queue Receiver

**Purpose:** Show the receiver side of message queues. The receiver opens the **existing** queue (created by the sender), then loops forever reading messages of type 1 as they arrive. This demonstrates the built-in blocking synchronization of message queues.

## 3.1 Full Source Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/types.h>
#include <sys/msg.h>

struct msgbuf {
    long mtype;
    char msgtxt[200];
};

int main(void) {
    struct msgbuf msg;
    int msgid;
    key_t key;

    key = ftok("message_send.c", 'b');
    if (key == -1) { perror("ftok"); exit(1); }

    msgid = msgget(key, 0644);
    if (msgid == -1) { perror("msgget"); exit(1); }

    printf("message_receive [INFO] Connected to Queue ID: %d\n",
       msgid);

    for (;;) {
        if (msgrcv(msgid, &msg, sizeof(msg), 1, 0) == -1) {
            perror("msgrcv");
            exit(1);
        }
        printf("message_receive [INFO] Message: %s", msg.msgtxt);
    }
    return 0;
}
```

Listing 2: Example 2: message_receiver.c – Message Queue Receiver

## 3.2 Line-by-Line Explanation

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
```

```
#include <sys/types.h>
#include <sys/msg.h>
```

**Lines 1–5 – Header files:**
Same headers as the sender. `string.h` is not included here because the receiver does not manipulate strings – it just prints whatever it receives. The four essential headers are the same: `stdio.h` (printf), `stdlib.h` (exit), `sys/ipc.h` (ftok, IPC flags), `sys/msg.h` (msgget, msgrcv).

```
struct msgbuf {
    long mtype;
    char msgtxt[200];
};
```

**Lines 7–10 – Message struct (redefined):**
The struct is defined **again** in the receiver file. In C, every source file that uses a struct must define it (or include a header that defines it). Since this is a separate `.c` file compiled independently, the struct definition is repeated. The struct must be **identical** in both sender and receiver – same field order, same field types, same sizes. If they differ, the receiver will misinterpret the received bytes.

```
    key = ftok("message_send.c", 'b');
    if (key == -1) { perror("ftok"); exit(1); }
```

**Lines 17–18 – Same ftok() as the sender:**
**Critical:** The receiver uses **exactly the same** file path (`"message_send.c"`) and project character (`'b'`) as the sender. This generates the **same key value**, which allows the receiver to find the same queue. If even one character differs, a different key is generated and the receiver connects to a non-existent or wrong queue.

```
    msgid = msgget(key, 0644);
    if (msgid == -1) { perror("msgget"); exit(1); }
```

**Lines 20–21 – `msgget()` without `IPC_CREAT`:**
**Key difference from the sender:** The receiver passes `0644` as the flags, **without** the `IPC_CREAT` flag.

- Without `IPC_CREAT`: the kernel only *opens* an existing queue. If no queue with this key exists, `msgget()` returns `-1` with error `ENOENT` ("No such file or directory").
- **Why not use IPC_CREAT here?** The sender "owns" the queue and is responsible for creating and deleting it. If the receiver accidentally created the queue, the sender might get confused about permissions or state.

**Important:** Always start the sender before the receiver, so the queue exists when the receiver tries to open it.

```
    printf("message_receive [INFO] Connected to Queue ID: %d\n",
        msgid);
```

**Line 23 – Confirm connection:**
Prints the queue ID to confirm the receiver successfully connected to the queue. The queue ID printed here should match the one printed by the sender. If they differ, something went wrong (different keys, different queues).

```
    for (;;) {
```

**Line 25 – Infinite loop `for(;;)`:**
`for(;;)` is identical to `while(1)` – an infinite loop. The receiver runs forever, continuously reading messages. In a real system, you would have a termination condition (e.g., a special "shutdown" message of a different type). The loop only ends if `msgrcv()` fails (e.g., the queue is deleted by the sender) and the `exit(1)` inside executes.

```
        if (msgrcv(msgid, &msg, sizeof(msg), 1, 0) == -1) {
            perror("msgrcv");
            exit(1);
        }
```

**Lines 26–29 – `msgrcv()` – Receive one message:**
`msgrcv(msgid, &msg, sizeof(msg), 1, 0)`:

- **Argument 1:** `msgid` – which queue to receive from.
- **Argument 2:** `&msg` – where to store the received message. The kernel copies bytes from its internal buffer into this struct.
- **Argument 3:** `sizeof(msg)` – maximum size of data to receive (208 bytes). The kernel will not copy more than this.
- **Argument 4:** `1` – **message type filter**. Only receive messages with `mtype == 1`. If you pass `0` here, the kernel returns the first message regardless of type. If you pass `-5`, it returns the lowest-type message $\leq 5$.
- **Argument 5:** `0` – flags. `0` means **blocking mode**: if no message of type 1 is in the queue, `msgrcv()` blocks (pauses the process) until one arrives. This is the built-in synchronization.
- **Returns:** number of bytes copied on success, `-1` on error.

**Theory: Blocking Behaviour and Synchronization in msgrcv():**

When `msgrcv()` is called and the queue is empty (no message of type 1):

1. The kernel puts the receiver process into a **wait/sleep state**.
2. The receiver consumes **zero CPU** while waiting (it is not in a busy loop).
3. When the sender calls `msgsnd()` with a type-1 message, the kernel wakes up the receiver.
4. The receiver copies the message and returns from `msgrcv()`.

This is **built-in synchronization** – no mutexes, semaphores, or `sleep()` calls needed. The kernel manages the waiting automatically.

```
        printf("message_receive [INFO] Message: %s", msg.msgtxt);
```

**Line 30 – Print the received message:**
After `msgrcv()` returns, `msg.msgtxt` contains the text sent by the sender (including the `'\n'` newline that `fgets()` added on the sender side). The `%s` format specifier prints the string.

**Note:** There is **no** `'\n'` at the end of this `printf()` format string. That is intentional – the newline was already included in `msg.msgtxt` by `fgets()` on the sender side.

## 3.3 Message Type Filtering – Visual Explanation

**How mtype filtering works in the queue:**

Imagine the queue contains these messages (in insertion order):

| Position | mtype | msgtxt |
|---|---|---|
| 1st | 1 | "hello\n" |
| 2nd | 2 | "priority message\n" |
| 3rd | 1 | "second message\n" |

`msgrcv(..., 1, 0)` – reads the **1st** message (mtype=1). Skips mtype=2 messages.
`msgrcv(..., 2, 0)` – reads the **2nd** message (mtype=2). Skips mtype=1 messages.
`msgrcv(..., 0, 0)` – reads the **1st** message regardless of type.
This is how priority-based communication works in message queues.

## 3.4 Expected Output – Both Programs Together

```
--- Terminal 1 (Sender) ---          --- Terminal 2 (Receiver) ---
message_send [INFO] Queue ID: 0      message_receive [INFO]
   Connected to Queue ID: 0
Enter messages (Ctrl+D to stop):
hello This is Yasir                  message_receive [INFO] Message:
    hello This is Yasir
this is second message               message_receive [INFO] Message:
    this is second message
^D
message_send [INFO] Queue deleted.   (msgrcv fails, receiver exits
   with error)
```

**Start order matters:**
The **sender must be started first** to create the queue. If the receiver starts first, `msgget(key, 0644)` will fail because the queue does not exist yet (no `IPC_CREAT` in the receiver). The error will be: `msgget: No such file or directory`.

# 4 Example 3 – Shared Memory Server (Writer)

**Purpose:** Show how to create a shared memory segment, attach it (map it into the process's virtual address space), write data directly into it using a pointer, and leave it alive for a client to read. This is the "server" (writer) side of shared memory IPC.

## 4.1 Full Source Code

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/shm.h>
5  #include <string.h>
6
7  int main() {
8      void *shared_memory;
9      char buff[100];
10     int shmid;
11
12     shmid = shmget((key_t)1122, 1024, 0666 | IPC_CREAT);
13     if (shmid == -1) { perror("shmget"); exit(1); }
14     printf("Key of shared memory is %d\n", shmid);
15
16     shared_memory = shmat(shmid, NULL, 0);
17     if (shared_memory == (void *)-1) { perror("shmat"); exit(1);
           }
18     printf("Process attached at %p\n", shared_memory);
19
20     printf("Enter some data to write to shared memory:\n");
21     read(0, buff, 100);
22     strcpy(shared_memory, buff);
23     printf("You wrote: %s\n", (char *)shared_memory);
24
25     return 0;
26 }
```

Listing 3: Example 3: shm_server.c – Shared Memory Server (Writer)

## 4.2 Header Files

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/shm.h>
#include <string.h>
```

**Lines 1–5 – Headers:**

- `stdio.h` – `printf()`, `perror()`.
- `stdlib.h` – `exit()`.
- `unistd.h` – `read()` system call (used to read from stdin using file descriptor 0).
- `sys/shm.h` – **The shared memory header**. Provides: `shmget()` (create/access), `shmat()` (attach), `shmdt()` (detach), `shmctl()` (control/delete). Also defines `IPC_CREAT` and `IPC_RMID` (though these are actually in `sys/ipc.h` which `sys/shm.h` includes automatically).
- `string.h` – `strcpy()` used to copy data into the shared memory segment.

## 4.3 Variable Declarations

```
    void *shared_memory;
    char buff[100];
    int shmid;
```

**Lines 8–10 – Variable declarations:**

- `void *shared_memory` – a **void pointer**. This will hold the virtual address in this process's address space where the shared memory segment is mapped. `void *` is used because the memory can hold any type of data. You cast it to the appropriate type when reading/writing.
- `char buff[100]` – a local buffer (on the stack) to temporarily hold user input before copying it into shared memory. This is **not** in shared memory – it is private to this process.
- `int shmid` – will hold the shared memory segment ID returned by `shmget()`. Used in all subsequent calls.

**Theory: What is a void pointer (`void *`)?**

A `void *` is a "generic pointer" – it can point to any type of data. When you need to read or write specific types through it, you **cast** it:

- `strcpy(shared_memory, buff)` – `strcpy()` accepts `void *` as destination (it internally treats it as `char *`).
- `(char *)shared_memory` – cast to `char *` to print as a string with `printf("%s")`.
- `(int *)shared_memory` – cast to `int *` if you stored integers.

`shmat()` returns `void *` because shared memory can hold any data.

## 4.4 Step 1 – shmget(): Create the Shared Memory Segment

```
    shmid = shmget((key_t)1122, 1024, 0666 | IPC_CREAT);
    if (shmid == -1) { perror("shmget"); exit(1); }
    printf("Key of shared memory is %d\n", shmid);
```

**Lines 12–14 – `shmget()` – Create the segment:**
`shmget((key_t)1122, 1024, 0666 | IPC_CREAT)`:

- **Argument 1:** `(key_t)1122` – the IPC key as a literal integer 1122, cast to `key_t`. This is simpler than using `ftok()` – both server and client just use the same hard-coded key. **Note:** using `ftok()` is safer in production because hard-coded keys can conflict with other programs on the system.
- **Argument 2:** `1024` – the size of the shared memory segment in **bytes**. Here, 1024 bytes (1 KB) are allocated. This is the maximum data that can be stored. The kernel will round this up to a multiple of the system's page size (usually 4096 bytes).
- **Argument 3:** `0666 | IPC_CREAT`:
  - `0666` – permissions: owner (rw), group (rw), others (rw). Both server and client need read+write access (6 = rw for each).
  - `IPC_CREAT` – create the segment if it does not exist.
- **Returns:** an integer **segment ID (shmid)** on success, `-1` on failure.

The `printf` confirms the ID. The client will get the same ID when it calls `shmget()` with the same key.

**shmget() behaviour summary:**

| Situation | Result |
|---|---|
| Segment with key 1122 does not exist + IPC_CREAT | Creates new 1024-byte segment, returns ID |
| Segment already exists + IPC_CREAT | Opens existing segment, returns existing ID |
| Segment does not exist, no IPC_CREAT | Returns -1 (ENOENT) |
| Insufficient permissions | Returns -1 (EACCES) |

## 4.5 Step 2 – shmat(): Attach the Segment

```
shared_memory = shmat(shmid, NULL, 0);
if (shared_memory == (void *)-1) { perror("shmat"); exit(1);
   }
printf("Process attached at %p\n", shared_memory);
```

**Lines 16–18 – `shmat()` – Map into address space:**
`shmat(shmid, NULL, 0)`:

- **Argument 1:** `shmid` – the segment ID from `shmget()`.
- **Argument 2:** `NULL` – the preferred virtual address to map the segment. Passing `NULL` tells the kernel to choose a suitable address automatically. This is almost always the right choice.
- **Argument 3:** `0` – flags. `0` means read-write access. Passing `SHM_RDONLY` would give read-only access.

- **Returns:** a `void *` pointer to the virtual address where the segment was mapped in this process's address space. On failure, returns `(void *)-1` (which is `0xFFFFFFFF...` on 64-bit systems).

**After shmat():** `shared_memory` is just a regular pointer. Reading and writing through it directly accesses the shared physical memory page. No system call needed for each read/write – this is what makes shared memory so fast.

**Error check:** `if (shared_memory == (void *)-1)` – note we compare to `(void *)-1`, not `NULL`. `shmat()` uses `-1` (not `NULL`) as its error indicator.

**Theory: What happens in the kernel during shmat()?**

Before `shmat()`: the shared memory segment exists in the kernel as physical memory pages, but this process's page table has no entry pointing to them.

After `shmat()`: the kernel adds entries to this process's **page table** pointing to the shared physical pages. Now when the CPU executes any instruction that accesses `shared_memory[0]`, it goes directly to the shared physical RAM. No kernel involvement per-access. This is why it is "zero copy" and very fast.

Both server and client call `shmat()` – both get **different virtual addresses** (e.g., `0x7108f3cff000` and `0x72b063a59000` as seen in the output) but they point to the **same physical memory**.

## 4.6 Step 3 – Writing Data into Shared Memory

```
printf("Enter some data to write to shared memory:\n");
read(0, buff, 100);
strcpy(shared_memory, buff);
printf("You wrote: %s\n", (char *)shared_memory);
```

**Lines 20–23 – Read from stdin and write to shared memory:**

**Line 20 – printf:**

Prompts the user to enter data. Output appears on the terminal.

**Line 21 – `read(0, buff, 100)`:**

- **Argument 1:** `0` – file descriptor 0 = `stdin` (keyboard). The `read()` system call reads raw bytes from a file descriptor. FD 0 is always stdin.
- **Argument 2:** `buff` – the destination buffer on the stack.
- **Argument 3:** `100` – maximum bytes to read.
- **Returns:** number of bytes actually read (including the newline if user pressed Enter).
- **Why use `read()` instead of `fgets()`?** Both work, but `read()` is a lower-level system call that directly uses the file descriptor. Using FD 0 explicitly emphasizes the relationship between file descriptors and I/O (from Lab 06).

**Line 22 – `strcpy(shared_memory, buff)`:**

- Copies the string from `buff` (local stack variable) into `shared_memory` (the shared segment).
- `strcpy()` copies until it finds a null terminator `'\0'` in `buff`.
- After this line, the shared physical memory page contains the user's input. The client process can now read it directly by accessing its own `shared_memory` pointer.

**Line 23** – `printf("You wrote: %s\n", (char *)shared_memory);`

- The `(char *)` cast converts the `void *` to `char *` so `printf("%s")` can print it as a string.
- This confirms the data was written correctly by reading it back through the pointer.

```
    // Note: We do NOT detach or remove here so the client can
       read it
    return 0;
```

**Lines 24–25 – Intentional: no shmdt() or shmctl(IPC_RMID) here:**
The server deliberately does **not** detach or remove the segment. This allows the client to run after the server and still read the data.

**What happens when `return 0` executes?**
The process exits. When a process exits:

- The kernel automatically detaches all shared memory segments this process had attached (effectively calls `shmdt()` for all of them).
- But the **segment itself is NOT deleted**. It stays in the kernel.
- The client can still attach to it and read the data.
- Only `shmctl(shmid, IPC_RMID, NULL)` (called by the client) actually removes the segment from the kernel.

**WARNING – Shared Memory Has NO Synchronization!**
In this example, the server and client work correctly only because we manually coordinate: run the server first (write data), then run the client (read data).

If both ran simultaneously and the client read before the server wrote, the client would get garbage data or zeros. In production, you must use **semaphores** to protect access:

1. Server acquires semaphore, writes, releases semaphore.
2. Client waits for semaphore, then reads.

# 5 Example 4 – Shared Memory Client (Reader)

**Purpose:** Show the client (reader) side of shared memory. The client accesses the **same physical memory** the server wrote to, reads the data directly through a pointer (zero kernel copies!), then properly detaches and removes the segment.

## 5.1 Full Source Code

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/shm.h>
5  #include <string.h>
6
7  int main() {
8      void *shared_memory;
9      int shmid;
10
11     shmid = shmget((key_t)1122, 1024, 0666);
12     if (shmid == -1) { perror("shmget"); exit(1); }
13     printf("Key of shared memory is %d\n", shmid);
14
15     shared_memory = shmat(shmid, NULL, 0);
16     if (shared_memory == (void *)-1) { perror("shmat"); exit(1);
           }
17     printf("Process attached at %p\n", shared_memory);
18
19     printf("Data read from shared memory is: %s\n",
20            (char *)shared_memory);
21
22     shmdt(shared_memory);
23     shmctl(shmid, IPC_RMID, NULL);
24     return 0;
25 }
```

Listing 4: Example 4: shm_client.c – Shared Memory Client (Reader)

## 5.2 Line-by-Line Explanation

```
        void *shared_memory;
        int shmid;
```

**Lines 8–9 – Variables:**
Same as the server. Note: the client does **not** need a `char buff[100]` local buffer because it reads directly from `shared_memory` without copying to a local variable first. This is the zero-copy advantage – data goes straight from the shared physical memory to the `printf()` call.

```
shmid = shmget((key_t)1122, 1024, 0666);
if (shmid == -1) { perror("shmget"); exit(1); }
printf("Key of shared memory is %d\n", shmid);
```

**Lines 11–13 – `shmget()` without `IPC_CREAT`:**
**Critical difference from server:** The client uses `0666` only, **without** `IPC_CREAT`.

- The client only *opens* the existing segment with key 1122. If the server has not run yet (segment does not exist), `shmget()` returns -1.
- The size argument `1024` must match or be less than the server's original size. On Linux, the kernel ignores the size when opening an existing segment (it uses the original allocated size).
- The returned `shmid` will be the **same integer** as the server's, because it is the same physical segment.

```
shared_memory = shmat(shmid, NULL, 0);
if (shared_memory == (void *)-1) { perror("shmat"); exit(1);
   }
printf("Process attached at %p\n", shared_memory);
```

**Lines 15–17 – `shmat()` on the client:**
Identical call to the server's `shmat()`. The kernel maps the same physical memory pages into the client's virtual address space.

**Key observation:** The `printf` will show a different virtual address than the server (e.g., `0x72b063a59000` vs server's `0x7108f3cff000`). This is **expected and normal**. Different virtual addresses, same physical memory. Writing to one is immediately visible through the other pointer.

```
printf("Data read from shared memory is: %s\n",
       (char *)shared_memory);
```

**Lines 19–20 – Reading directly from shared memory:**
`(char *)shared_memory` casts the `void *` to `char *` so `printf("%s")` can print it as a null-terminated string.

**THIS IS THE ZERO-COPY READ:**
No `read()` system call. No kernel involvement. The CPU simply loads bytes from the virtual address `shared_memory`, which the page table maps directly to the shared physical RAM where the server wrote the data. The data transfer from server to client involves **zero copies**. This is why shared memory is the fastest IPC mechanism.

After this line executes, `"This is Example of shm\n"` (or whatever the server wrote) is printed to the terminal.

## 5.3 Step 4 – shmdt(): Detach the Segment

```
shmdt(shared_memory);
```

**Line 22 – `shmdt()` – Detach from address space:**
`shmdt(shared_memory)`:

- **Argument:** `shared_memory` – the pointer returned by `shmat()`. This is the virtual address to unmap.
- **What it does:** The kernel removes the page table entries that mapped the shared segment into this process's address space. After this call, `shared_memory` is a dangling pointer – accessing it causes undefined behaviour (likely a segmentation fault).
- **Returns:** 0 on success, -1 on failure.
- **The physical shared memory is NOT deleted** by `shmdt()`. It just removes this process's mapping. Another process could still attach to it.

**Always detach before deleting.**

## 5.4 Step 5 – shmctl(): Remove the Segment

```
        shmctl(shmid, IPC_RMID, NULL);
        return 0;
```

**Lines 23–24 – `shmctl(IPC_RMID)` – Delete the segment:**
`shmctl(shmid, IPC_RMID, NULL)`:

- **Argument 1:** `shmid` – the segment to delete.
- **Argument 2:** `IPC_RMID` – the command: "Remove this IPC resource." The kernel marks the segment for deletion. The physical memory is freed once all processes that have it attached call `shmdt()` or exit. Since we already called `shmdt()`, the memory is freed immediately.
- **Argument 3:** `NULL` – only needed for `IPC_STAT` (get info) and `IPC_SET` (set permissions) commands. For `IPC_RMID`, it is ignored; pass `NULL`.
- **Returns:** 0 on success, -1 on failure.

After this call, the shared memory segment no longer exists in the kernel. The server segment is cleaned up.

**WARNING – Shared Memory Persists Like Message Queues!**
If the client crashes before calling `shmctl(IPC_RMID, ...)`, the segment stays in kernel memory indefinitely (until reboot or manual removal).

**To list all shared memory segments:** `ipcs -m`
**To delete an orphaned segment:** `ipcrm -m <shmid>`

Always call `shmctl(shmid, IPC_RMID, NULL)` in your cleanup code.

## 5.5 Server vs Client – Complete Step Comparison

**Shared Memory – All 5 Steps Side by Side:**

| Step | Action | Server (Writer) | Client (Reader) |
|---|---|---|---|
| 1 | Create/Access | `shmget(1122, 1024, 0666\|IPC_CREAT)` | `shmget(1122, 1024, 0666)` |
| 2 | Attach | `shmat(shmid, NULL, 0)` | `shmat(shmid, NULL, 0)` |
| 3 | Use | `strcpy(ptr, data)` (write) | `printf("%s", ptr)` (read) |
| 4 | Detach | (omitted in example, but use `shmdt()`) | `shmdt(ptr)` |
| 5 | Remove | (client is responsible) | `shmctl(id, IPC_RMID, NULL)` |

## 5.6 Expected Output – Both Programs Together

```
--- Terminal 1 (Server) ---
Key of shared memory is 0
Process attached at 0x7108f3cff000
Enter some data to write to shared memory:
This is Example of shm
You wrote: This is Example of shm

--- Terminal 2 (Client) ---
Key of shared memory is 0              <- Same shmid as server
Process attached at 0x72b063a59000     <- Different virtual address!
Data read from shared memory is: This is Example of shm   <- Zero
   copy!
```

# 6 IPC API Complete Reference

## 6.1 Message Queue API

| Call | Header | Purpose | Returns |
|---|---|---|---|
| `ftok(path, id)` | `<sys/ipc.h>` | Generate unique key from file inode + project char | `key_t` or -1 |
| `msgget(key, flags)` | `<sys/msg.h>` | Create/open message queue | queue ID or -1 |
| `msgsnd(qid, msg, sz, 0)` | `<sys/msg.h>` | Send a message struct to the queue | 0 or -1 |
| `msgrcv(qid, buf, sz, type, 0)` | `<sys/msg.h>` | Receive message of given type; blocks if empty | bytes read or -1 |
| `msgctl(qid, IPC_RMID, NULL)` | `<sys/msg.h>` | Delete the message queue | 0 or -1 |

## 6.2 Shared Memory API

| Call | Header | Purpose | Returns |
|---|---|---|---|
| `shmget(key, size, flags)` | `<sys/shm.h>` | Create/access shared memory segment | shmid or -1 |
| `shmat(shmid, NULL, 0)` | `<sys/shm.h>` | Attach: map segment into address space | `void *` or `(void*)-1` |
| `shmdt(ptr)` | `<sys/shm.h>` | Detach: unmap segment from address space | 0 or -1 |
| `shmctl(id, IPC_RMID, NULL)` | `<sys/shm.h>` | Remove shared memory segment from kernel | 0 or -1 |

## 6.3 Useful Shell Commands for IPC Debugging

| Command | Purpose |
|---|---|
| `ipcs` | List ALL IPC resources (queues, shared memory, semaphores) |
| `ipcs -q` | List only message queues |
| `ipcs -m` | List only shared memory segments |
| `ipcrm -q <msgid>` | Remove orphaned message queue by ID |
| `ipcrm -m <shmid>` | Remove orphaned shared memory segment by ID |

# 7 Most Likely Quiz Questions – Summary

**Based on Lab Manual 07 content and the 4 code examples, these are the most important points for your quiz:**

1. **Why do we need ftok()?**
   Both sender and receiver must use the same IPC key to find the same queue or memory segment. `ftok()` generates a unique key from a file's inode number + project character. Both programs MUST use the same arguments.

2. **What must be the first field of a message struct?**
   A `long` integer named (by convention) `mtype`. It MUST be positive ($> 0$). The kernel uses it to identify message types.

3. **What does the type argument in msgrcv() do?**
   It filters which message to read. Type=1: only read type-1 messages. Type=0: read the first message regardless of type. This enables priority-based IPC.

4. **What is the difference between msgget() in sender vs receiver?**
   Sender uses `IPC_CREAT` flag (creates the queue if not exist). Receiver does NOT use `IPC_CREAT` (only opens existing queue).

5. **Why is shared memory the fastest IPC?**
   Zero data copies. The kernel maps the same physical RAM into both processes' virtual address spaces. No kernel involvement per-access. Pipes and message queues require 2 copies (user→kernel, kernel→user).

6. **What does shmat() return?**
   A `void *` pointer to the virtual address where the segment is mapped. On error it returns `(void *)-1` (NOT NULL).

7. **Do server and client get the same virtual address from shmat()?**
   NO. They get different virtual addresses but they point to the same physical memory pages. Both can read and write the same data through different pointers.

8. **What is the difference between shmdt() and shmctl(IPC_RMID)?**
   `shmdt()` only detaches (unmaps) the segment from this process's address space. The segment still exists. `shmctl(IPC_RMID)` actually deletes the segment from the kernel.

9. **Why must shared memory use semaphores in production?**
   It has NO built-in synchronization. Two processes writing simultaneously cause a race condition – corrupted, unpredictable data.

10. **What happens if you don't call msgctl(IPC_RMID) or shmctl(IPC_RMID)?**
    The queue/segment stays in the kernel permanently (even after the program exits) until the machine reboots or you manually remove it with `ipcrm`.

11. **What does msgrcv() do if the queue is empty?**
    It **blocks** (the process sleeps, consuming no CPU) until a message of the requested type arrives. This is the built-in synchronization of message queues (no semaphores needed).

12. **Why is (void *)-1 used as the error value for shmat()?**
Because `NULL` (address 0) is a valid memory address on some architectures. Using `-1` (all bits set, i.e., the highest possible address) avoids ambiguity. Always check `== (void *)-1` for shmat errors.